LexAid

AI-Powered Multilingual Legal Aid Platform for Marginalized Communities

Master Plan v4 — Consolidated, Start to End

Registered under: Vishwakarma Institute of Technology, FF No. 180, Dept. of Computer Engineering, Group BE-A35, Guide: Prof. Rahul Dagade.

This document supersedes all earlier partial plans (v1 initial plan, v2 gap analysis, v3 differentiators). It is the single, current source of truth for scope, architecture, data sourcing, and the phase roadmap.

Document date: 23 September 2026

1. Problem Statement & Vision

India has 1.4 billion citizens but roughly 1.3 million practising lawyers, mostly concentrated in metro cities. Getting legal guidance today means one of three things: paying Rs. 500–5,000 for a consultation, travelling 40–80 km to a DLSA office and losing a day's wages, or trying to make sense of English-language legal text that most citizens can't read. General AI chatbots don't fix this — they answer from training data that may be outdated or hallucinated, don't cite the law sections they rely on, and can't reason about a specific physical document a citizen is holding.

LexAid is a free, multilingual, RAG-grounded legal information platform: it answers in the user's own language, cites the exact Act and Section behind every claim, and lets users photograph a real document (FIR, notice, contract) and ask questions about its specific content. It directly targets SDG 16.3 (equal access to justice), and secondarily SDG 1, 5, and 10 (poverty, gender equality, reduced inequalities).

1.1 Why this over ChatGPT/Claude — the real differentiators

Grounded, cited answers are necessary but not sufficient on their own — general assistants can already produce fluent legal-sounding answers. Two features were deliberately chosen because they require something a general chat assistant structurally doesn't have:

- **Guided intake (triage wizard):** a first-time, possibly distressed user often doesn't know the right question to ask. A short structured flow (safety check → situation category → what happened → what the user wants) does what a paralegal actually does, and feeds clean, structured signals into the emergency-detection and referral-engine logic — rather than trying to parse them out of free text.
- **WhatsApp channel:** the target user is on WhatsApp, not opening a web app or ChatGPT. A second interface via Meta's WhatsApp Cloud API removes the install/discovery barrier entirely — this is a distribution advantage no general AI assistant offers for this audience.

> Note: two other candidate differentiators — a document-generation engine (auto-filled RTI/consumer-complaint/notice drafts) and a case-file/deadline-tracking engine — were scoped in an earlier plan revision and then deliberately dropped from the build. They remain fully specified in the project history if revisited later; this master plan reflects the current, reduced scope of two differentiators.

2. Requirements

The laptop constraint (no local model-serving capability) shapes the whole stack: nothing heavy runs locally. LLM inference happens on Groq's hosted API; the only local compute is a small CPU embedding model and in-browser OCR.

2.1 Accounts to set up (all free-tier)

Service	Purpose
Groq Console	LLM inference — free tier (~15 req/min), matches the FF No.180 zero-cost requirement exactly
Supabase	Postgres DB, Auth, file storage
Vercel + Render/Railway	Hosting frontend + backend, free tier
Meta for Developers (WhatsApp Cloud API)	WhatsApp channel — free tier, ~1,000 service conversations/month
Hugging Face	Download the local multilingual embedding model

2.2 Local machine requirements

- Python 3.10+, Node.js 18+, Git.
- No GPU needed anywhere — the embedding model and Tesseract.js-adjacent tooling run on CPU.
- ~2–3 GB free disk for dependencies and the local embedding model.

2.3 Decisions locked so far

Decision	Locked choice
Legal scope	9 statutes (see Section 5) — matches FF No.180 exactly, not reduced from the original plan
Language scope	Hindi, Marathi, Tamil, Telugu, Bengali, English — direct LLM generation, no translation pipeline
Frontend	Next.js (SPA) — required by the architecture diagram; Streamlit dropped, doesn't meet the spec
OCR	In-browser only (Tesseract.js) — document image never reaches the server, on the web channel
Voice input	Browser-native Web Speech API
Labour Laws	Industrial Disputes Act 1947 + Minimum Wages Act 1948 (older acts, still enforced; new Labour Codes still rolling out)
Rent Control	Maharashtra Rent Control Act 1999 (state-specific by design)
IPC/CrPC vs BNS/BNSS	v1 covers IPC/CrPC only; BNS/BNSS cross-mapping deferred to a later addition

2.4 Still open

- Confirm the VIIT registration form is the correct one for this build (its listed team — Suraj Khaire, Suyash Kadake, Avishkar Ladhe, Snehal Wadhane — doesn't include Yash).
- Keep Bhashini as a fallback/QA layer for multilingual output, or drop it entirely in favor of pure LLM generation?

3. System Architecture

3.1 Flow

- **Guided intake:** user starts with the triage wizard (safety check → category → situation → goal) on web or WhatsApp.
- **Emergency path:** if the safety check flags immediate danger, the system responds with helpline numbers immediately, bypassing retrieval/generation entirely.
- **Referral path:** the situation category feeds a specialist referral engine that names the correct forum (Consumer Forum, Labour Court, Family Court, Rent Control Court) and points to DLSA free aid.
- **Document layer (web only):** an uploaded document is OCR'd entirely in-browser (Tesseract.js) — the image never reaches the server, only the extracted text does.
- **Document layer (WhatsApp):** a photo sent via WhatsApp necessarily transits the server for OCR — processed immediately, never persisted, with explicit consent messaging before upload.
- **Retrieval:** the query is embedded with a multilingual embedding model and matched against a vector store of chunked, cited statute text (+ any uploaded document text).
- **Generation:** Groq LLM answers directly in the user's language from the retrieved chunks, citing Act + Section; if no reliable match is found, it says so rather than guessing.
- **Response:** rendered with clickable citations, disclaimer appended, logged (anonymized) for evaluation.

3.2 Tech stack

Layer	Choice
LLM	Groq — openai/gpt-oss-120b primary, openai/gpt-oss-20b fallback
Embeddings	multilingual-e5-small or LaBSE — local, CPU
Vector store	Self-hosted open-source (Chroma/Qdrant) — kept open-source in production too, to stay Rs. 0
OCR (web)	Tesseract.js — in-browser only
OCR (WhatsApp)	Server-side, image deleted immediately after text extraction
Voice input	Browser-native Web Speech API
Backend	FastAPI — REST API, ORM, migrations, auth middleware, rate limiting
Frontend	Next.js SPA — chat UI, intake wizard, OCR preview, language selector, session history
WhatsApp	Meta WhatsApp Cloud API via FastAPI webhook, reusing the core backend
Auth/DB/Storage	Supabase (Postgres + Auth + Storage)
Hosting	Vercel (frontend) + Render/Railway (backend), free tiers

3.3 Security & privacy

- JWT auth with configurable expiry; passwords hashed (bcrypt/argon2), never stored in plaintext.
- Role-based authorization: citizen vs. institutional admin.
- HTTPS-only; parameterized ORM queries; rate limiting on all routes.
- Registration requires only email + password — no name, phone, address, or Aadhaar.
- Session data auto-purged after 24 hours unless the user explicitly saves it.
- Secrets (LLM API key, JWT secret, DB connection string) via environment variables only.
- WhatsApp sessions are a separate, clearly-disclosed privacy tier — tied to a phone number, unlike the web app's email-only registration — never silently held to the stricter web standard.

3.4 Mandatory disclaimer (verbatim, session start + every response)

"LexAid provides legal information based on publicly available Indian law. This is not legal advice and does not create a lawyer-client relationship."

3.5 Sustainability

- Six independently-upgradeable modules: RAG Pipeline, LLM Integration, OCR, Authentication, Analytics, Corpus Management.
- Incremental corpus updates — an amended law triggers re-chunking/re-embedding of only the affected sections, never a full rebuild.
- Version-controlled DB migrations; manifest files pin embedding model, LLM model, and corpus version.

4. SDG Mapping & Cost

SDG	How LexAid addresses it
SDG 1 — No Poverty	Removes Rs. 500–5,000 consultation costs for BPL/daily-wage families
SDG 5 — Gender Equality	PWDVA + POCSO in corpus; emergency detection routes domestic-abuse signals to helplines
SDG 10 — Reduced Inequalities	Regional-language support, voice input, in-browser OCR remove literacy/language/geography barriers
SDG 16 — Peace, Justice, Strong Institutions	Free, cited, AI-grounded legal information — UN Target 16.3

Cost: Rs. 0 for development, testing, and demonstration — every component is either open-source or sits within a free tier (Groq's free tier alone covers ~15 req/min, which is sufficient for dev and demo traffic).

5. Legal Corpus & Data Sources

5.1 The 9 statutes — verified working sources (as of 23 Sept 2026)

Act	Verified source
Indian Penal Code, 1860	thc.nic.in (Telangana High Court mirror)
Code of Criminal Procedure, 1973	thc.nic.in (Telangana High Court mirror)
RTI Act, 2005	cic.gov.in (Central Information Commission)
Consumer Protection Act, 2019	ncdrc.nic.in — verified working
PWDVA, 2005	Not yet confirmed — several gov mirrors timed out during checking; retry from your own browser/IP
Maharashtra Rent Control Act, 1999	Govt of India S3WaaS mirror — verified working
Industrial Disputes Act, 1947	Govt of India S3WaaS mirror — verified working
Minimum Wages Act, 1948	clc.gov.in (Chief Labour Commissioner) — verified working
POCSO Act, 2012	AdvocateKhoj — full text, section-by-section — verified working
Negotiable Instruments Act, 1881 (§138–142)	Govt of India S3WaaS — verified working

Note: India Code (indiacode.nic.in) itself is unreliable — it sits behind an Akamai edge network that frequently throttles or errors even from a real browser. Prefer High Court, ministry, regulator, or Govt S3WaaS mirrors going forward.

5.2 Acquisition method

- Download official PDFs directly — never LLM-paraphrased before ingestion.
- Extract with **pdftotext -layout** to preserve section boundaries.
- Chunk by Section (not fixed token windows); tag each chunk with Act/Chapter/Section/cross-references.
- A manifest file per Act records source URL, retrieval date, checksum, last-verified date — supports incremental re-indexing when a law is amended.
- Corpus stays in English; multilingual embeddings handle cross-lingual retrieval — only the LLM's answer is generated in the user's language.

5.3 Supporting data for the referral engine & emergency detection

Need	Source
Consumer complaint e-filing	e-Daakhil — edaakhil.nic.in
RTI e-filing (central govt only)	rtionline.gov.in (state RTI portals are separate — Maharashtra's not yet identified)
DLSA / free legal aid locator	nalsa.gov.in
Women's helpline (181) / NCW	ncw.gov.in — helplines page
Childline (1098) — POCSO cases	Official Childline site
Police emergency	100 / 112 — hardcode, no lookup needed

5.4 Evaluation golden set (team-authored, not scraped)

- ~10–15 questions per Act × 9 Acts ≈ 100–130 questions, each with a manually-verified expected citation and model-answer summary — built by the team reading the Acts directly, since this set is what catches LLM hallucination and can't itself rely on an LLM.
- A subset (~30) gets manual multilingual paraphrases to test cross-lingual retrieval specifically.
- Ideally sanity-checked by Prof. Dagade or a DLSA/NGO contact for realism.

5.5 Synthetic test documents (for OCR testing)

3–5 in-house-built mock documents (FIR copy, eviction notice, consumer complaint acknowledgment) — never a real citizen's document. These double as demo assets for Review 1/2 and the final presentation.

6. Full Phase Roadmap (12 Phases)

Phase	Goal	Key deliverables	Status
0. Setup & Planning	Scope locked, accounts, repo	This master plan signed off; Groq/Supabase/GitHub/Meta accounts; golden eval set drafted	In progress
1. Core RAG Engine	Grounded Q&A over all 9 legal sources	Corpus collected/chunked (Section 5); multilingual embeddings + local vector DB; Groq LLM w/ citation-enforcing + refusal prompt	Data acquisition underway
2. Multilingual Generation	Direct in-language answers, 6 languages	Language detection; direct-generation prompting; eval across all 6 languages	Not started
3. In-Browser Document Intelligence	OCR entirely client-side (web)	Tesseract.js integration; image never transmitted; doc text merged into retrieval	Not started
4. Guided Intake, Referral & Emergency Detection	Structured triage → safe routing	Triage wizard; emergency classifier; specialist referral engine (forum mapping + DLSA/e-Daakhil/RTI Online pointers)	Not started
5. Web Application	Full SPA per architecture diagram	Next.js chat UI incl. intake wizard, OCR preview, language selector, voice input, session history	Not started
6. WhatsApp Channel	Reach users where they already are	Meta WhatsApp Cloud API webhook; shared backend reuse; explicit image-privacy consent flow	Not started
7. Auth, Roles & Admin Dashboard	Citizen vs institutional admin	JWT auth, hashed passwords, RBAC; institutional analytics dashboard (anonymized aggregates only)	Not started
8. Reliability & Guardrails	Production-safe	Rate limiting, retry/backoff, exact disclaimer text, 24h session purge, structured logging	Not started
9. Evaluation & Testing	Prove correctness across corpus + languages	100+ golden set across 9 acts/6 languages; faithfulness/citation-accuracy scoring	Not started
10. Deployment & Hardening	Ship at Rs. 0	CI/CD; free-tier hosting; secrets via env vars; cost/usage monitoring	Not started
11. Polish & Submission	Present to guide/review panel	Demo script (incl. WhatsApp demo), README, architecture diagram, Review 1/2 reports, pitch materials	Not started

7. Immediate Next Steps

- Confirm the two open decisions in Section 2.4 (registration-form roster check; Bhashini fallback or not).
- Finish Phase 1 data acquisition: confirm a working PWDVA source; start section-by-section chunking on the 9 Acts already downloaded.
- Start authoring the golden evaluation set in parallel — it doesn't depend on the pipeline being built.
- Once corpus + eval set are ready, begin the core RAG integration against Groq (Phase 1 build).